

# What are functions?

Python function is a block of code that performs a specific task & can be reused multiple times throughout a program.

block -

```
if <condition>:  
    _____  
    _____  
    _____
```

```
else:  
    _____  
    _____  
    _____
```

addition

```
def Sum():  
    _____  
    _____  
    _____  
    _____
```

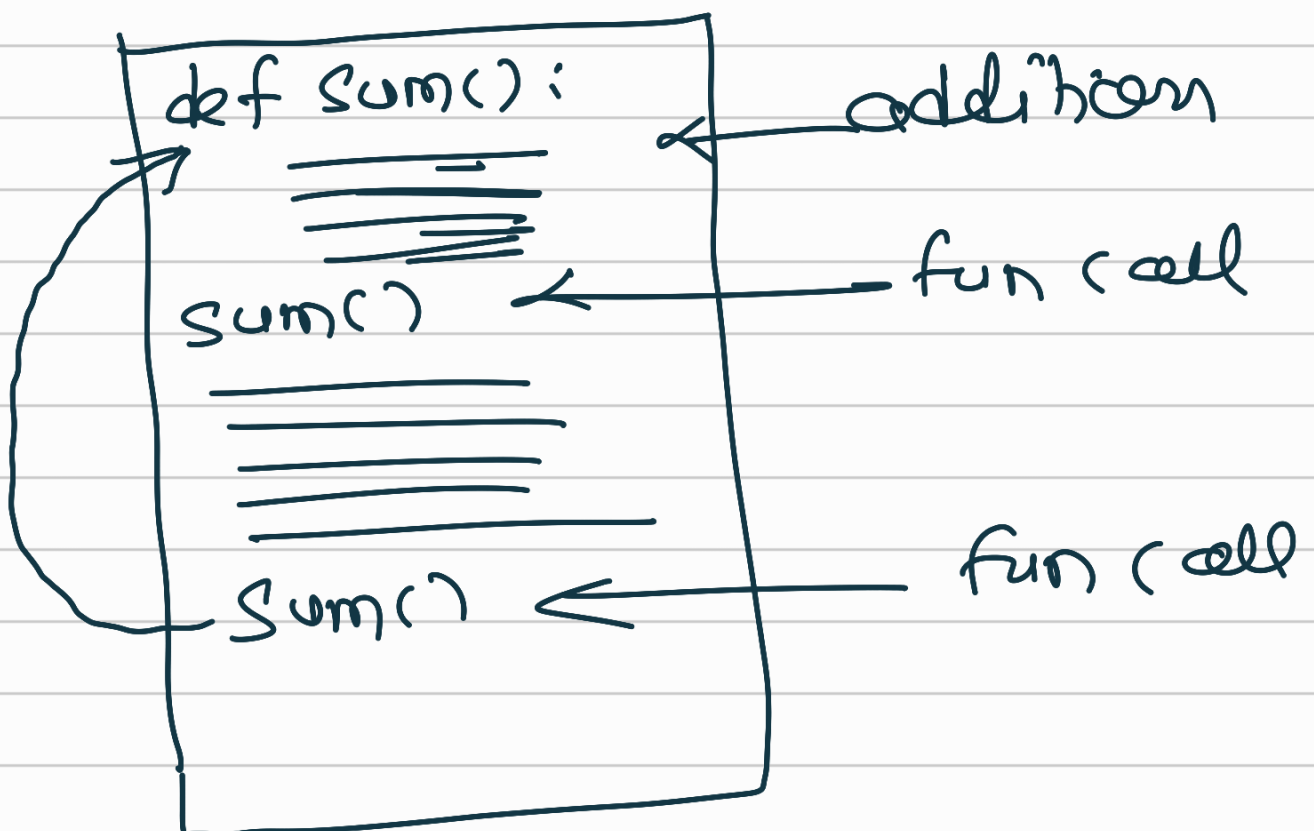
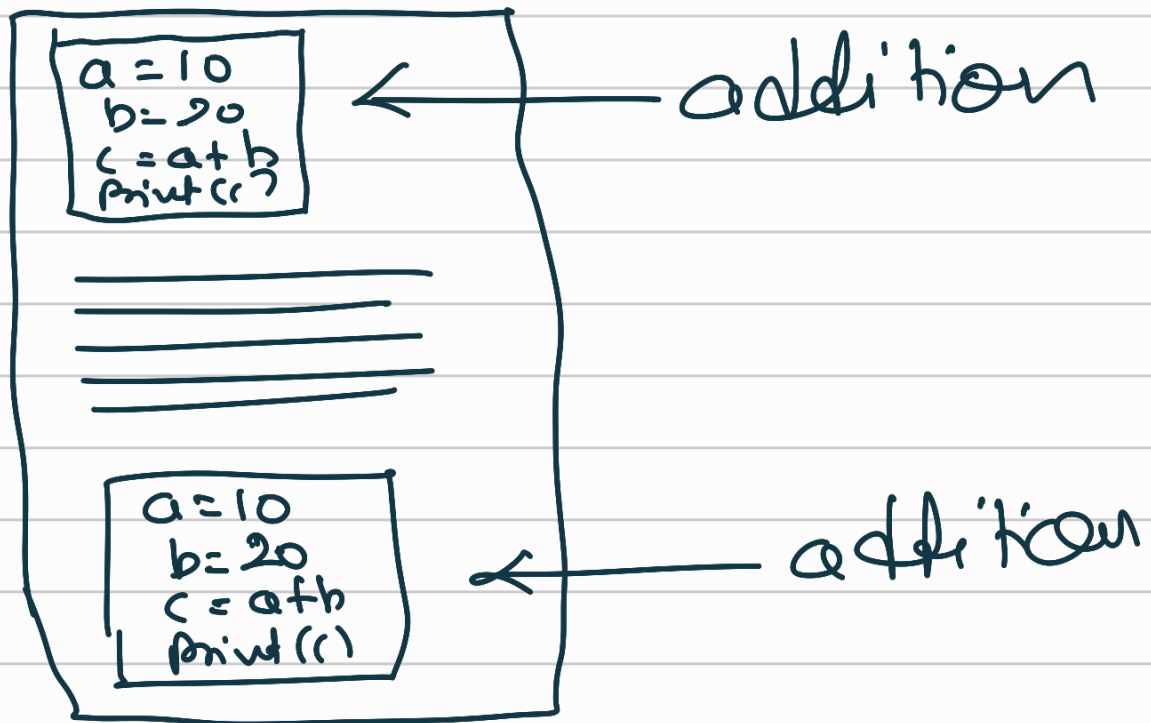
100 1 function

100 5 function

10 function

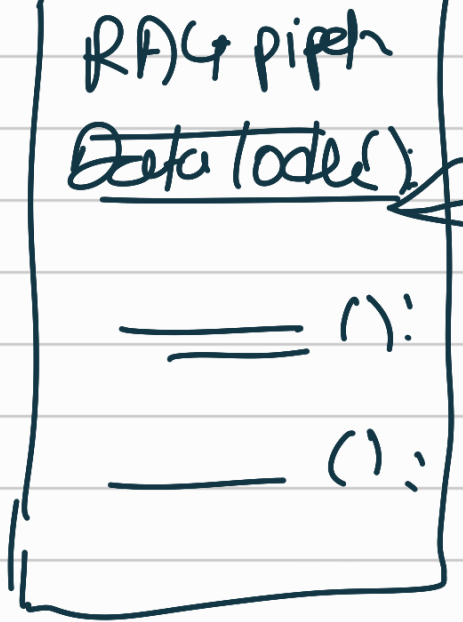
## Advantages of function -

① Code reusability - we can reuse the code multiple times.



Req. PY

Req pipeline

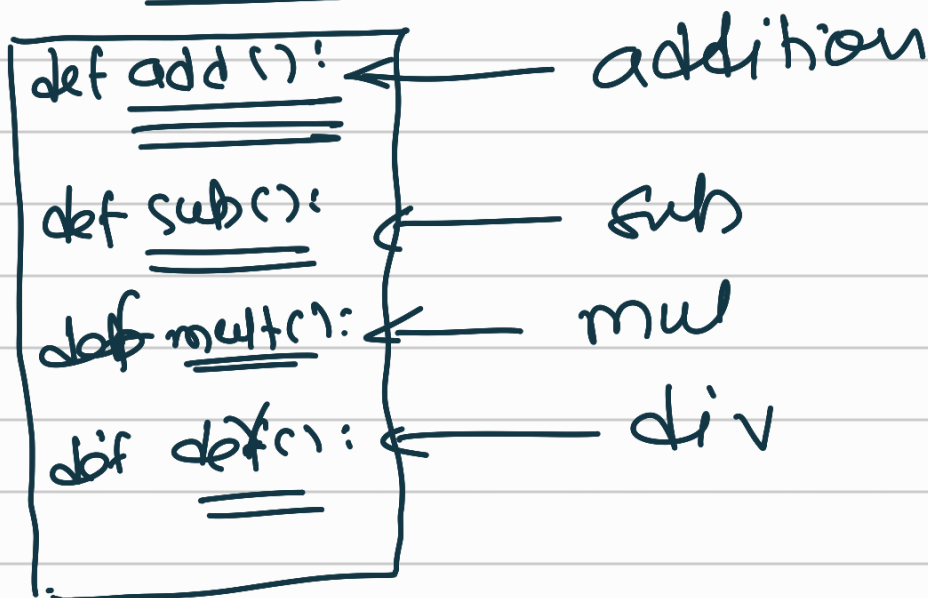


6 month

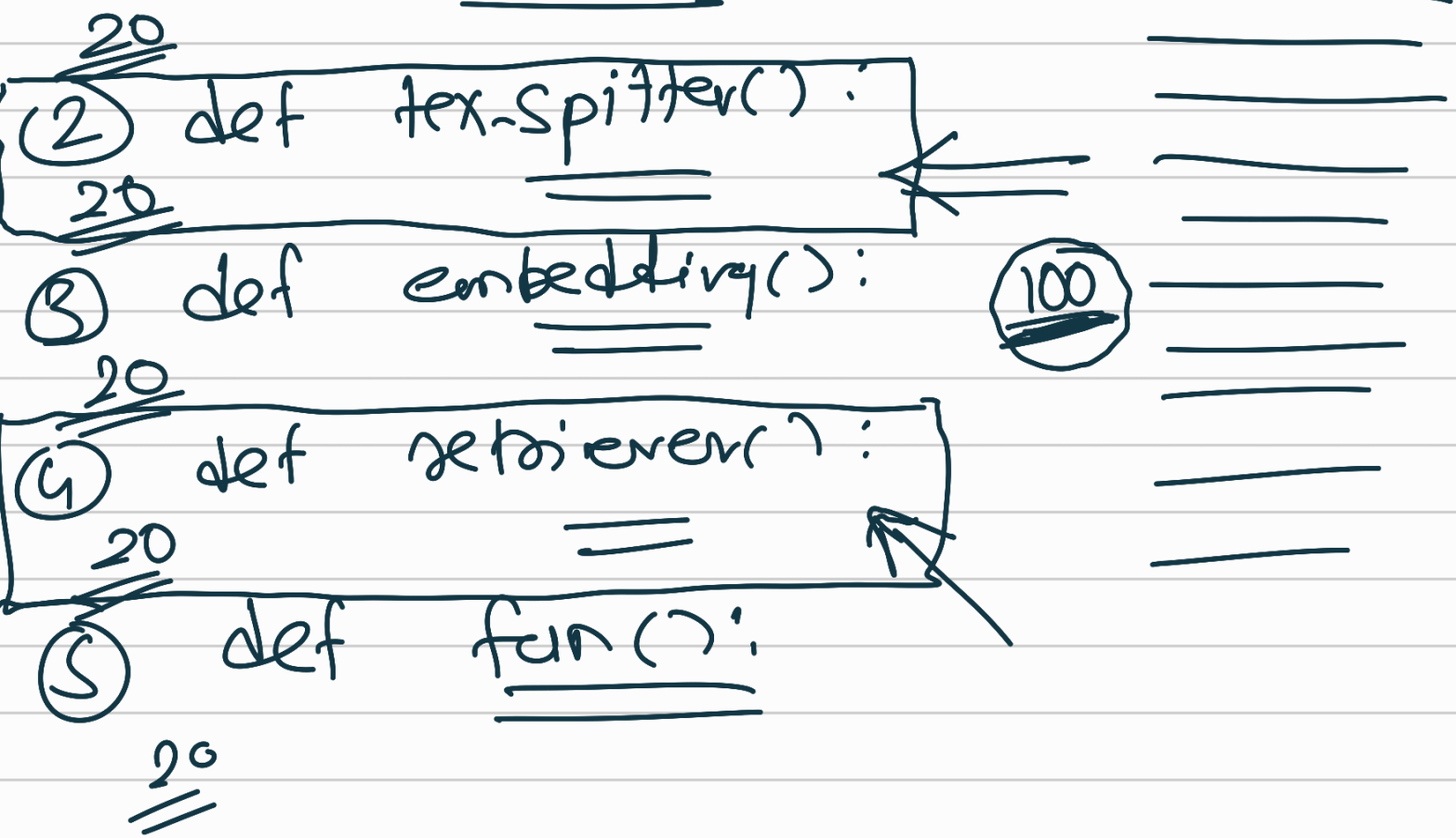


after 6 month

② Modularity - we convert complex program into smaller manageable code. every function performs a specific task.



① def doc\_loader(): def RAG():



3) ease of maintenance:  
easy to modify &  
update functions

4) testing & debugging:  
easy for testing &  
debugging

5) Improved readability - we

Can provide meaningful names to the function, so that it is easy to understand.

def <fun\_name> :

eg. def calculate\_area :

"""

=====

input : ←

output : →

"""

← doc  
String

=====

There are two types of functions

1) Built in functions - readily available

eg. len(), min(), max(),

Sum(), any(), all(),

sorted(), reversed(),

type(), id(),

print(), input(),

str(), list(), tuple(), ~~set()~~, dict()

eg. list1 = [10, 20, 30]

print(len(list1))

o/p - 3

2] User-defined functions -

we use def keyword to define a

function, then we write <fun.name>

& then we provide parenthesis  
& color. This is called

function definition

Syntax:- def <fun\_name>():

\_\_\_\_\_  
\_\_\_\_\_  
\_\_\_\_\_

} fun  
definition

fun.name() ← fun call

To call the function we

write function name & parenthesis

eg. 

|  |                            |              |
|--|----------------------------|--------------|
|  | def calculate_area():      | } fun<br>def |
|  | _____                      |              |
|  | _____                      |              |
|  | _____                      |              |
|  | ← tab →                    |              |
|  | calculate_area()           | → fun call   |
|  | <del>calclate_area()</del> | → fun call   |

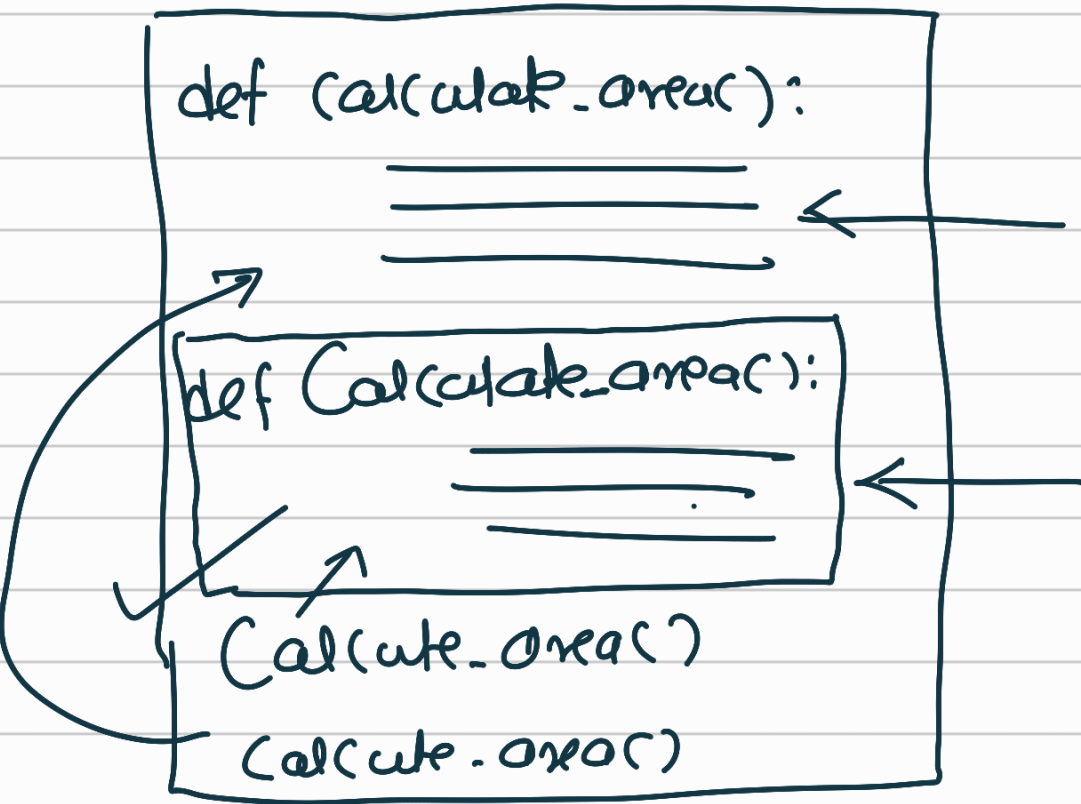
Convention for function names

① use lowercase

② join the multiple words  
with underscore ( \_ )

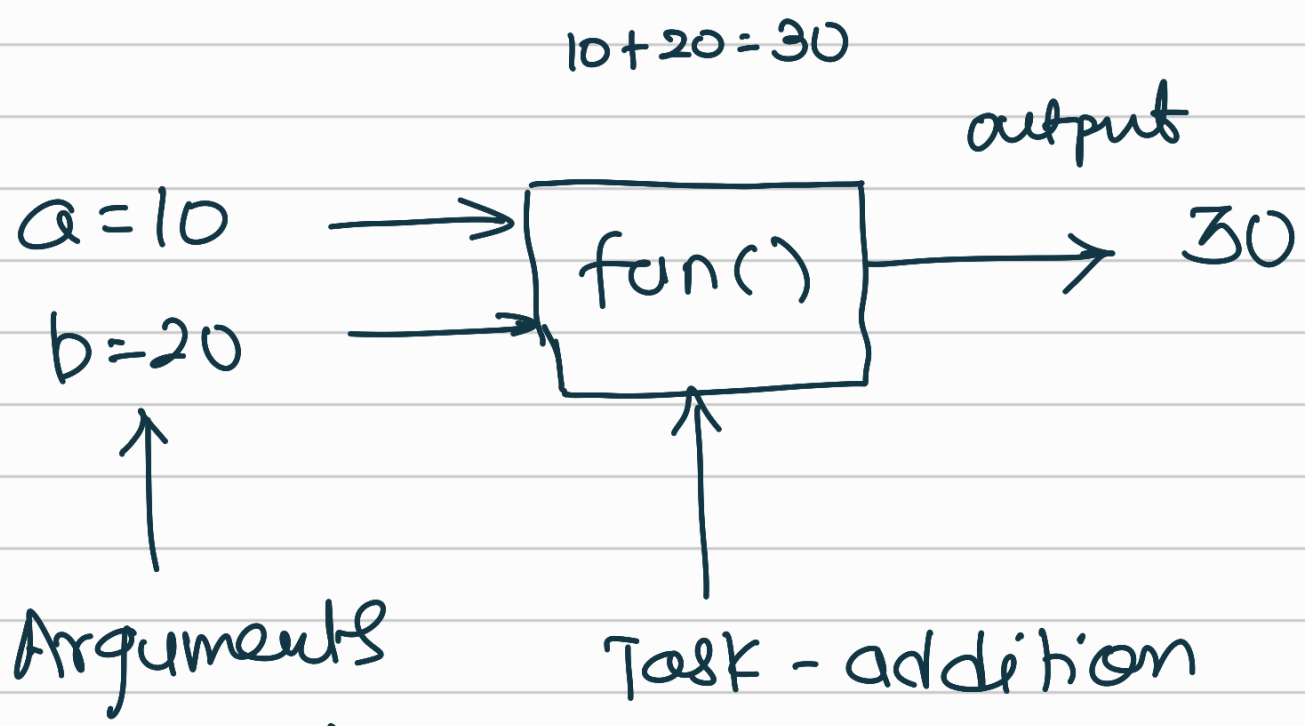
Calculate Area() X

calculate\_area() ✓



if two functions having the same name, then latest function will be executed

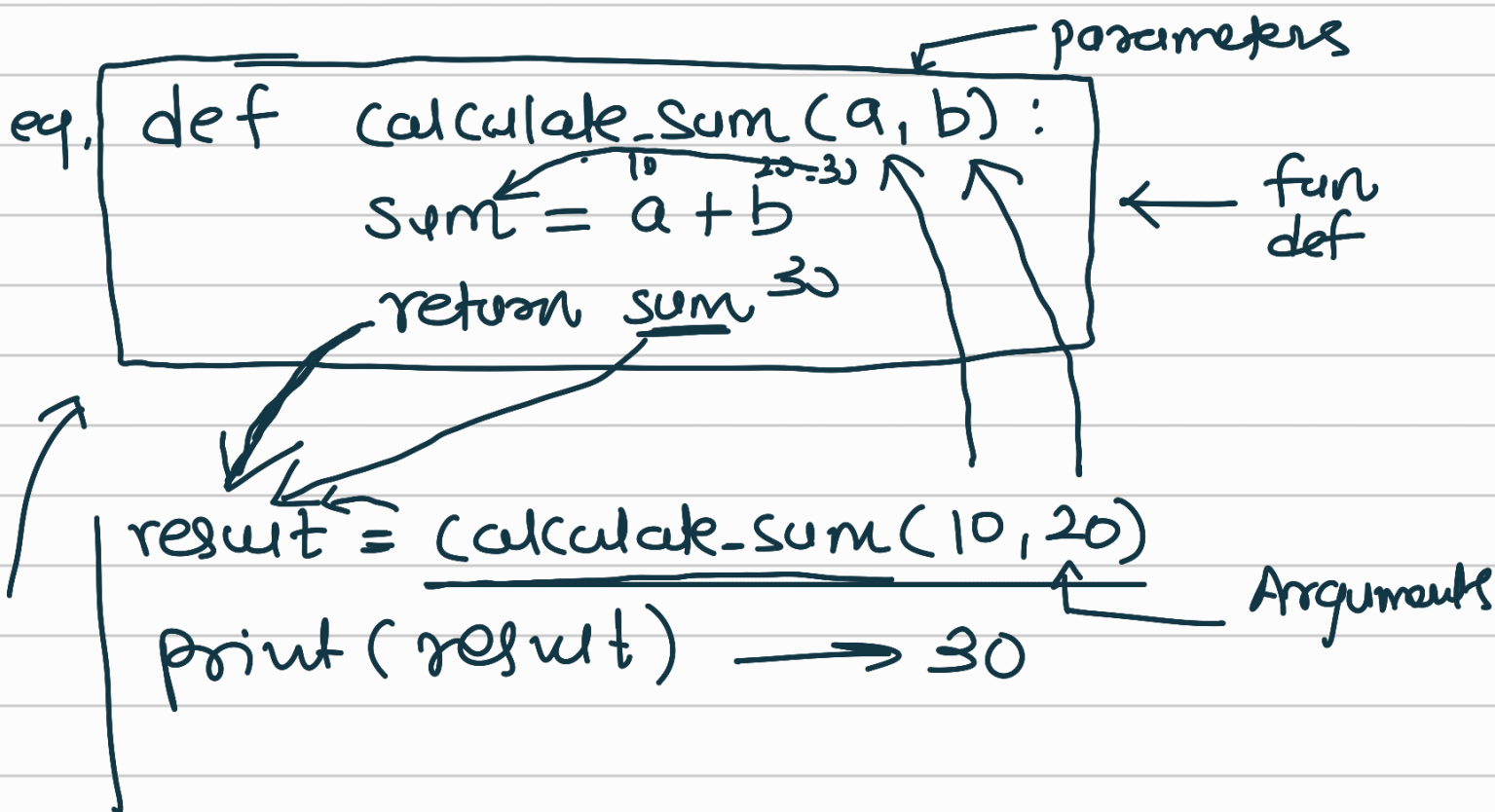
### Communication with function.



(input)

- 1) we pass some arguments to a function — input
- 2) function returns some value — output

In this way we communicate with functions.



return keyword — return keyword.  
return the result of the function,

\* program's control goes to the function call.

if we don't use return in the function definition, it returns None by default.

## Types of arguments

- ① Positional arguments -
- ② keyword arguments -
- ③ variable length positional arguments.
- ④ variable length keyword arguments
- ⑤ default arguments

### 1] Positional arguments -

- 1) This is the most common type.
- 2) we pass the value on the basis

of positions & assign it to the  
parameters

3) & we make sure that passing  
value in the correct position

eg. `def greet (name, age):`

`print ("my name is", name,  
"and age is", age)`

`greet ("rahul", 20)`

①

②

argument

`greet ( 20 , "rahul" )`

2] keyword arguments -

1) we can pass arguments in any  
order.

2) we match the keywords with the  
parameters, used in the function.

parameter

```
eg. def greet ( name, age ) :  
    print ( "my name is:", name,  
            "and age is", age)  
  
greet (name = "rahul", age = 20)  
greet (age = 20, name = "rahul")
```

### 3] Default arguments +

1) we can provide default value to a parameter, and if we do not pass a value then it will consider this default value.

eg. without default argument.

```
def greet (name, age):  
    print ("my name is", name,  
           "and age is", age)
```

`greet("rahul")`

o/p - error - ~~Type Error~~

eg. with default arguments.

```
def greet (name = "Akshay", age = 30) :  
    print ("my name is", name,  
          "and age is", age)
```

`greet("rahul")` → my name is rahul & age is 30

`greet()` → my name is akshay & age is 30

`greet("rahul", 25)` → my name is rahul & age is 25

`greet(age = 15)` → my name is Akshay & age is 15